SOC Detection & Incident Response Report

Project 3 — Purple-Team Web Application Attack & Defense

SIEM	Wazuh 4.14 (manager: soc-waz)
Monitored host	ubuntu-dvwa · agent 002 · 192.168.64.8
Source of attack	192.168.64.7 (Kali Linux)
Analyst	Sawhard Poojari
Role	SOC Analyst / Blue Team
Incident date	26 October 2026
Classification	Confidential — Lab Use Only

1. Executive Summary

This report presents the defensive side of Project 3: how a live, multi-stage attack against a monitored web server was detected, investigated, and (in one stage) automatically contained using Wazuh. The offensive details are in the companion Penetration Test Findings Report. Here the focus is the SOC workflow — what the analyst saw, how the investigation was conducted, which detections fired, and where the detection coverage has honest gaps.

Outcome. Every post-reconnaissance stage of the attack generated alerts in Wazuh, the full kill chain was reconstructed from a single source-IP pivot, and the SSH brute force was **automatically blocked by Active Response with no human intervention**. Two coverage gaps were identified and are addressed with concrete recommendations (§7).

> **Detections at a glance**
> Brute force (rule 5712, auto-blocked) · SQL injection (31106) · Web shell written to disk (554, File Integrity Monitoring) · Remote command execution (31514). One gap: network reconnaissance was not alerted (host-based SIEM limitation).

2. Monitoring Environment

Component	Configuration
SIEM platform	Wazuh 4.14 (all-in-one: manager, indexer, dashboard)
Manager	soc-waz
Monitored endpoint	ubuntu-dvwa (agent ID 002), 192.168.64.8
Log sources collected	Apache access/error logs, /var/log/auth.log, UFW firewall log
File Integrity Monitoring	/var/www/html — realtime, report_changes enabled
Active Response	firewall-drop on SSH brute force (rule 5712), 300s timeout
Attacker (as seen in logs)	192.168.64.7

The log collection, the realtime FIM directory, and the Active Response binding were all configured as part of building this lab; they are what makes the detections below possible.

3. Investigation Workflow

A real analyst does not start from rule numbers — they start from something anomalous and pivot. This investigation is presented the way it actually unfolds in the console.

3.1 Triage — a spike of activity

Reviewing the last 24 hours of events on the monitored host surfaced an unusually large volume of alerts concentrated in a short window. That volume alone is the trigger to investigate.

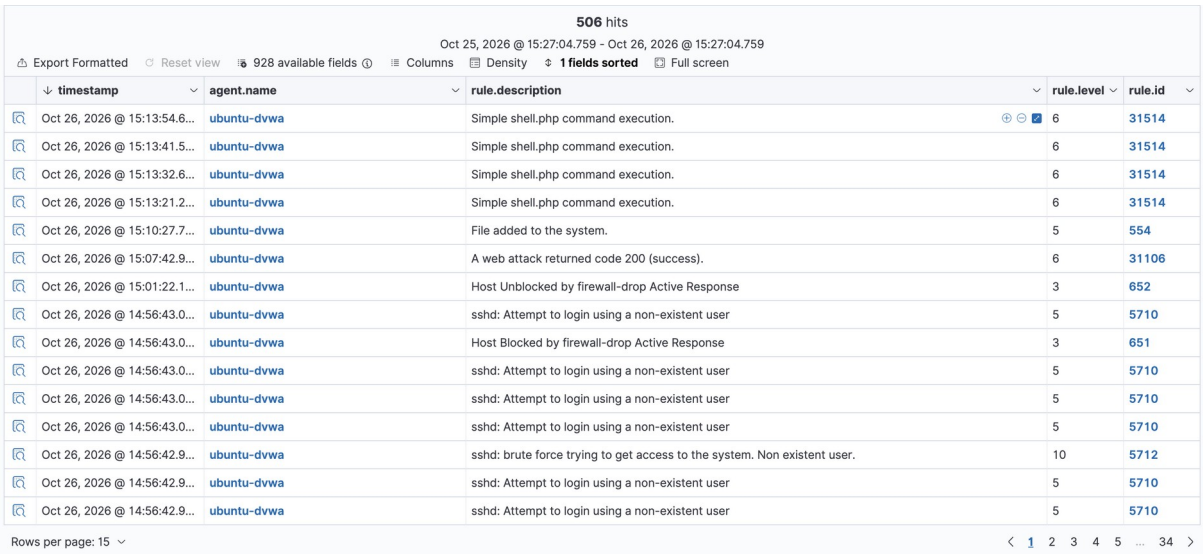


Figure 3.1 — Broad view: 506 events on ubuntu-dvwa, clustered into a short burst. The descriptions already hint at a chain — brute force, web attack, file added, command execution.

3.2 Scoping — pivot on the source IP

The natural next move is to attribute the activity. Filtering on the source address seen in the alerts, data.srcip:192.168.64.7, scopes the noise down to a single actor — and that one actor accounts for the entire attack chain.

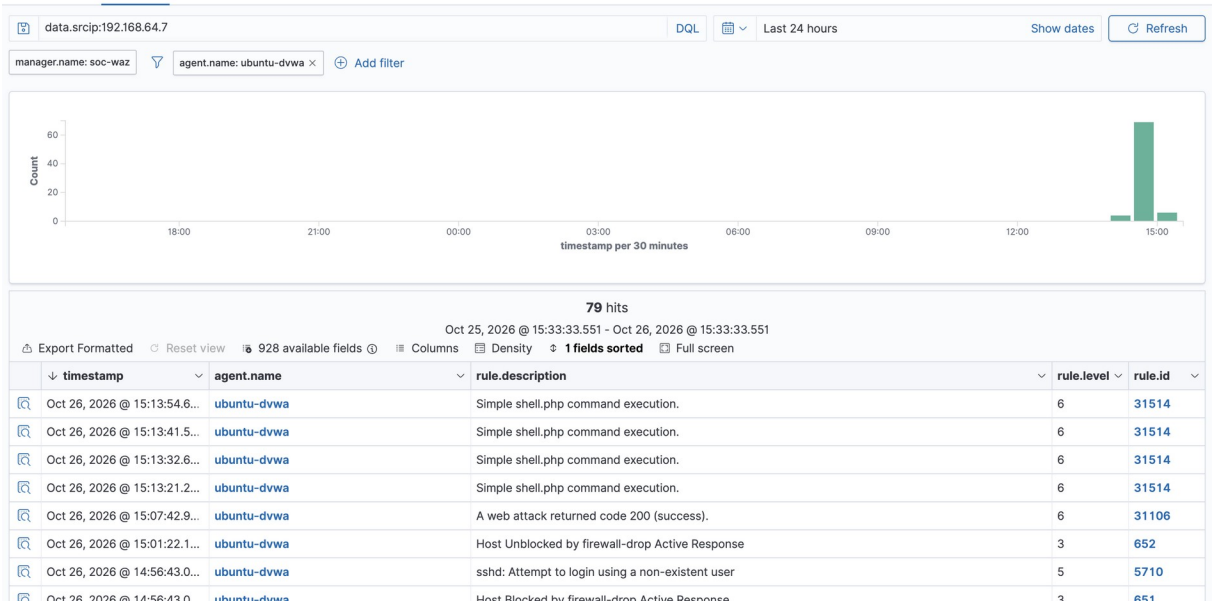


Figure 3.2 — Pivoted to data.srcip:192.168.64.7. 79 events, one source, the whole kill chain in order: command execution → web attack → host blocked/unblocked → brute force. The incident is now scoped to one attacker.

Why search this way

In practice an analyst uses two complementary styles. Entity / behaviour pivots (source IP, rule.groups:authentication_failures) are for discovery — scoping an incident when you don't yet know its shape. Rule-ID filters are for precision — once the dashboard surfaces a known detection, filtering

> by its rule ID isolates a single clean detection to verify and document. This investigation used IP pivoting to scope the incident, then rule-ID filtering to confirm and evidence each stage below.

4. Detections by Attack Stage

Reading the scoped timeline oldest-first reconstructs exactly what the attacker did. Each stage below pairs the detection with the evidence captured from Wazuh.

4.1 SSH Brute Force — Rule 5712 (MITRE T1110)

The incident opens with a burst of failed SSH logins against a non-existent user. Wazuh's correlation escalates repeated failures (rule 5710) into a single high-severity brute-force alert (rule 5712, level 10), tagged Credential Access / Brute Force.

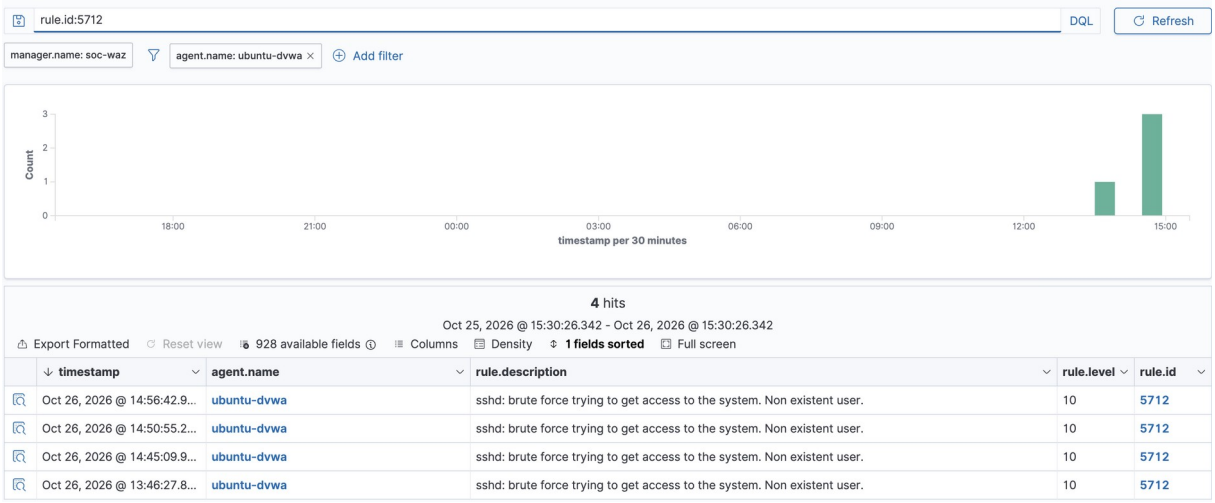


Figure 4.1a — rule.id:5712: repeated brute-force alerts from the same source.

Table JSON

t	_index	wazuh-alerts-4.x-2026.10.26
t	agent.id	002
t	agent.ip	192.168.64.8
t	agent.name	ubuntu-dvwa
t	data.srcip	192.168.64.7
t	data.srcuser	admin
t	decoder.name	sshd
t	decoder.parent	sshd
t	full_log	2026-10-26T14:54:11.258009+05:30 spector sshd-session[35019]: Failed password for invalid user admin from 192.168.64.7 port 54302 ssh2
t	id	1793006802.356493
t	input.type	log
t	location	/var/log/auth.log
t	manager.name	soc-waz
t	predecoder.program_name	sshd-session
t	predecoder.timestamp	2026-10-26T14:54:11.258009+05:30
t	rule.description	sshd: Attempt to login using a non-existent user
#	rule.firedtimes	34
t	rule.gdpr	IV_35.7.d, IV_32.2
t	rule.gpg13	7.1
t	rule.groups	syslog, sshd, authentication_failed, invalid_login

Figure 4.1b — Underlying failed-login event (rule 5710): "Failed password for invalid user admin from 192.168.64.7", decoded from /var/log/auth.log.

t	_index	wazuh-alerts-4.x-2026.10.26
t	agent.id	002
t	agent.ip	192.168.64.8
t	agent.name	ubuntu-dvwa
t	data.srcip	192.168.64.7
t	data.srcport	54302
t	data.srcuser	admin
t	decoder.name	sshd
t	decoder.parent	sshd
t	full_log	2026-10-26T14:54:12.233284+05:30 spector sshd-session[35019]: Disconnected from invalid user admin 192.168.64.7 port 54302 [preauth]
t	id	1793006802.357627
t	input.type	log
t	location	/var/log/auth.log
t	manager.name	soc-waz
t	predecoder.program_name	sshd-session
t	predecoder.timestamp	2026-10-26T14:54:12.233284+05:30
t	previous_output	n from 192.168.64.7 port 54304 Oct 26 09:24:08 spector sshd-session[35019]: Invalid user admi n from 192.168.64.7 port 54302 2026-10-26T14:54:08.950322+05:30 spector sshd-session[35020]: Invalid user admin from 192.168.64.7 port 54304 2026-10-26T14:54:08.944075+05:30 spector sshd-session[35019]: Invalid user admin from 192.168.64.7 port 54302
t	rule.description	sshd: brute force trying to get access to the system. Non exis tent user.

Figure 4.1c — Alert detail: data.srcip 192.168.64.7, srcuser admin, full raw log preserved for evidence.

#	rule.firedtimes	5
#	rule.frequency	8
t	rule.gdpr	IV_35.7.d, IV_32.2
t	rule.groups	syslog, sshd, authentication_failures
t	rule.hipaa	164.312.b
t	rule.id	5712
#	rule.level	10
◔	rule.mail	false
t	rule.mitre.id	T1110
t	rule.mitre.tactic	Credential Access
t	rule.mitre.technique	Brute Force
t	rule.nist_800_53	SI.4, AU.14, AC.7
t	rule.pci_dss	11.4, 10.2.4, 10.2.5
t	rule.tsc	CC6.1, CC6.8, CC7.2, CC7.3
🗓	timestamp	Oct 26, 2026 @ 14:56:42.960

Figure 4.1d — The same alert auto-mapped to MITRE T1110 (Credential Access / Brute Force) and to PCI-DSS, HIPAA and NIST 800-53 controls.

Analyst note. the rich context — source IP, targeted user, raw log, MITRE technique, and compliance mappings — all arrive on a single alert, which is exactly what makes triage fast.

4.2 Automated Containment — Active Response (Rules 651 / 652)

This is the key defensive result. The moment the brute-force rule fired, Wazuh Active Response executed the firewall-drop command on the endpoint, blocking the attacker's IP automatically — no analyst in the loop. After the configured 300-second timeout the IP was unblocked, and because the attacker was still trying, the block fired again. This block/unblock cycle is visible both in the SIEM and as the ~5-minute stalls in the attacker's own Medusa output.

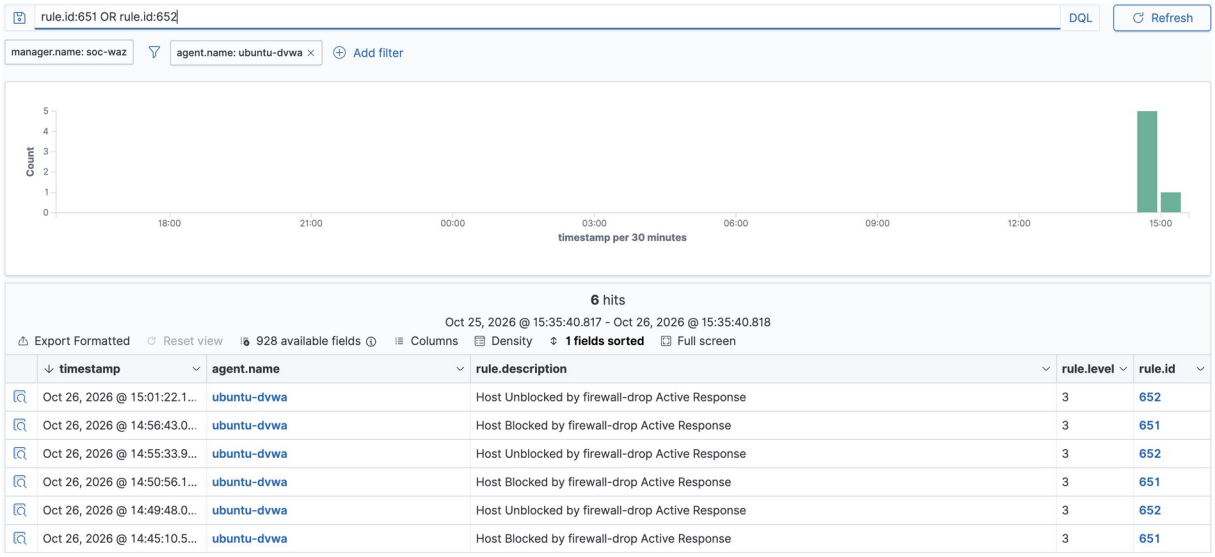


Figure 4.2a — rule.id:651 OR 652: the automated block/unblock cycle. Host Blocked (651) and Host Unblocked (652) alternate as the attacker persists and the timeout expires.

t	_index	wazuh-alerts-4.x-2026.10.26
t	agent.id	002
t	agent.ip	192.168.64.8
t	agent.name	ubuntu-dvwa
t	data.command	add
t	data.origin.module	wazuh-execd
t	data.origin.name	node01
t	data.parameters.alert.agent.id	002
t	data.parameters.alert.agent.ip	192.168.64.8
t	data.parameters.alert.agent.name	ubuntu-dvwa
t	data.parameters.alert.data.srcip	192.168.64.7
t	data.parameters.alert.data.srcport	34524
t	data.parameters.alert.data.srcuser	admin
t	data.parameters.alert.decoder.name	sshd
t	data.parameters.alert.decoder.parent	sshd
t	data.parameters.alert.full_log	Oct 26 09:11:40 spector sshd-session[34946]: Invalid user admin from 192.168.64.7 port 34524
t	data.parameters.alert.id	1793006109.309250

Figure 4.2b — Active Response detail: command "add", origin module wazuh-execd, targeting srcip 192.168.64.7 / srcuser admin.

t	data.parameters.alert.rule.mitre.tactic	Credential Access
t	data.parameters.alert.rule.mitre.technique	Brute Force
t	data.parameters.alert.rule.nist_800_53	SI.4, AU.14, AC.7
t	data.parameters.alert.rule.pci_dss	11.4, 10.2.4, 10.2.5
t	data.parameters.alert.rule.tsc	CC6.1, CC6.8, CC7.2, CC7.3
t	data.parameters.alert.timestamp	2026-10-26T09:15:09.914+0000
t	data.parameters.extra_args	
t	data.parameters.program	active-response/bin/firewall-drop
t	data.srcip	192.168.64.7
t	data.version	1
t	decoder.name	ar_log_json
t	decoder.parent	ar_log_json
t	full_log	2026/10/26 14:41:41 active-response/bin/firewall-drop on":1,"origin":{"name":"node01","module":"wazuh-execd"},"comma nd":"add","parameters":{"extra_args":[],"alert":{"timestam p":"2026-10-26T09:15:09.914+0000","rule":{"level":10,"descript ion":"sshd: brute force trying to get access to the system. No n existent user.","id":"5712","mitre":{"id":["T1110"],"tacti c":["Credential Access"],"technique":["Brute Force"]},"frequen
t	id	1793006110.313709
t	input.type	log
t	location	/var/ossec/logs/active-responses.log

Figure 4.2c — The executed program: active-response/bin/firewall-drop, carrying the originating brute-force alert (rule 5712, T1110) as justification.

Automated defence confirmed working

Unlike a purely detective control, this stopped the attack in progress. The firewall-drop response triggered on rule 5712, blocked 192.168.64.7 at the host firewall, and demonstrably forced the attacker to pause for the full timeout before each retry. Detection and response, end to end, with zero human action.

4.3 SQL Injection — Rule 31106 (MITRE T1190)

Once the attacker pivoted to the web application, Apache access logs shipped to Wazuh captured the injection. Rule 31106 (“a web attack returned code 200”) fired — significant because the 200 status means the malicious request succeeded rather than being rejected.

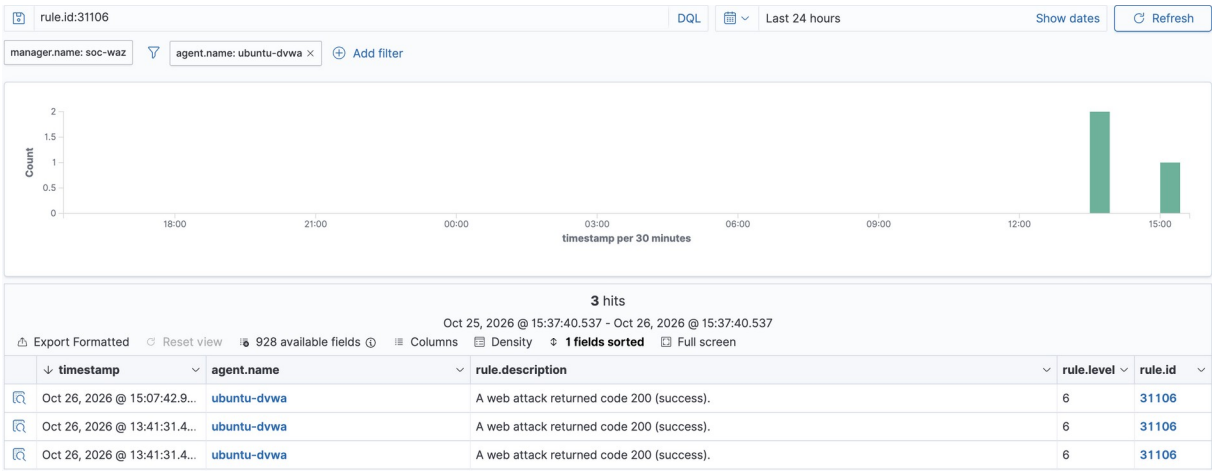


Figure 4.3a — rule.id:31106: successful web-attack responses recorded.

t	_index	wazuh-alerts-4.x-2026.10.26
t	agent.id	002
t	agent.ip	192.168.64.8
t	agent.name	ubuntu-dvwa
t	data.id	200
t	data.protocol	GET
t	data.srcip	192.168.64.7
t	data.url	/dvwa/vulnerabilities/sqli/?id=+++1%27+UNION+SELECT+user%2C+password+FROM+users--+-&Submit=Submit
t	decoder.name	web-accesslog
t	full_log	192.168.64.7 - - [26/Oct/2026:15:06:06 +0530] "GET /dvwa/vulnerabilities/sqli/?id=+++1%27+UNION+SELECT+user%2C+password+FROM+users--+-&Submit=Submit HTTP/1.1" 200 2009 "http://192.168.64.8/dvwa/vulnerabilities/sqli/?id=+++1%27+OR+%271%27%3D%271&Submit=Submit" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
t	id	1793007462.373823
t	input.type	log
t	location	/var/log/apache2/access.log
t	manager.name	soc-waz
t	rule.description	A web attack returned code 200 (success).
#	rule.firedtimes	1
t	rule.gdpr	IV_35.7.d
t	rule.groups	web, accesslog, attack

Figure 4.3b — Alert detail: data.url shows the UNION SELECT user,password FROM users payload; the full request, source IP and user-agent are preserved from /var/log/apache2/access.log.

Analyst note. the exact injection payload is readable straight off the alert, so an analyst can confirm what data the attacker targeted (the users table) without leaving the console.

4.4 Web Shell Dropped — Rule 554 (File Integrity Monitoring)

The attacker then uploaded a PHP web shell. Because /var/www/html is under realtime File Integrity Monitoring, the new file was detected the instant it touched disk — rule 554, “File added to the system”, decoded by syscheck_new_entry.

t	_index	wazuh-alerts-4.x-2026.10.26
t	agent.id	002
t	agent.ip	192.168.64.8
t	agent.name	ubuntu-dvwa
t	decoder.name	syscheck_new_entry
t	full_log	File '/var/www/html/dvwa/hackable/uploads/shell.php' added Mode: realtime
t	id	1793007627.375889
t	input.type	log
t	location	syscheck
t	manager.name	soc-waz
t	rule.description	File added to the system.
#	rule.firedtimes	1
t	rule.gdpr	II_5.1.f
t	rule.gpg13	4.11
t	rule.groups	ossec, syscheck, syscheck_entry_added, syscheck_file
t	rule.hipaa	164.312.c.1, 164.312.c.2
t	rule.id	554
#	rule.level	5
◕	rule.mail	false
t	rule.nist_800_53	SI.7

Figure 4.4 — FIM alert: '/var/www/html/dvwa/hackable/uploads/shell.php' added, Mode: realtime. The malicious file is caught at write time, before it is ever used.

Analyst note. FIM is the control that catches the payload itself. Even if the upload request had been missed, the file landing on disk raises an independent alert — defence in depth.

4.5 Remote Command Execution — Rule 31514 (MITRE T1059)

Finally the attacker used the shell. Each request to shell.php carried the command in the URL, so Apache logged it and Wazuh raised rule 31514 (“shell.php command execution”) — once per command.

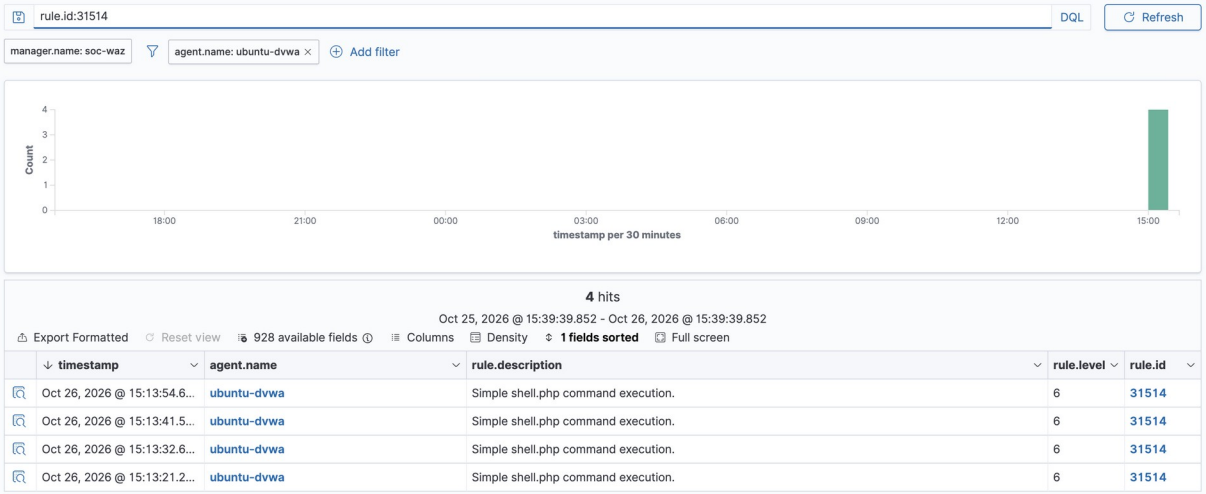


Figure 4.5a — rule.id:31514: four command-execution alerts, matching the four commands run.

t	_index	wazuh-alerts-4.x-2026.10.26
t	agent.id	002
t	agent.ip	192.168.64.8
t	agent.name	ubuntu-dvwa
t	data.id	200
t	data.protocol	GET
t	data.srcip	192.168.64.7
t	data.url	/dvwa/hackable/uploads/shell.php?cmd=cat%20/etc/passwd
t	decoder.name	web-accesslog
t	full_log	192.168.64.7 - - [26/Oct/2026:15:12:48 +0530] "GET /dvwa/hackable/uploads/shell.php?cmd=cat%20/etc/passwd HTTP/1.1" 200 3320 "-" "curl/8.20.0"
t	id	1793007834.379422
t	input.type	log
t	location	/var/log/apache2/access.log
t	manager.name	soc-waz
t	rule.description	Simple shell.php command execution.
#	rule.firedtimes	4
t	rule.gdpr	IV_35.7.d
t	rule.groups	web, appsec, attack
t	rule.id	31514
#	rule.level	6

Figure 4.5b — Alert detail: data.url = /dvwa/hackable/uploads/shell.php?cmd=cat /etc/passwd. The executed command is fully visible to the analyst.

Analyst note. the command is captured because it travelled in the URL query string. This is also where an honest detection limitation lives — see §7.

5. Incident Timeline

Reconstructed from the scoped alerts (all times IST, 26 Oct 2026):

Time	Stage	Detection	Response
~14:24 / 14:39	Reconnaissance (Nmap)	Not alerted (see §7)	UFW logged scan
14:45–14:56	SSH brute force	Rule 5712 (level 10)	Auto-block (651/652)
~15:06–15:07	SQL injection	Rule 31106	Detected
15:10	Web shell uploaded	Rule 554 (FIM realtime)	Detected at write
15:12–15:13	Command execution (RCE)	Rule 31514 (×4)	Detected

6. Dashboard Overview

The Wazuh dashboards summarise the incident at a glance: a single monitored host generating a sharp burst of alerts, and the attacker's techniques auto-classified against MITRE ATT&CK.



Figure 6a — Alert evolution: all activity concentrated on ubuntu-dvwa in a short window — the signature of an active incident.

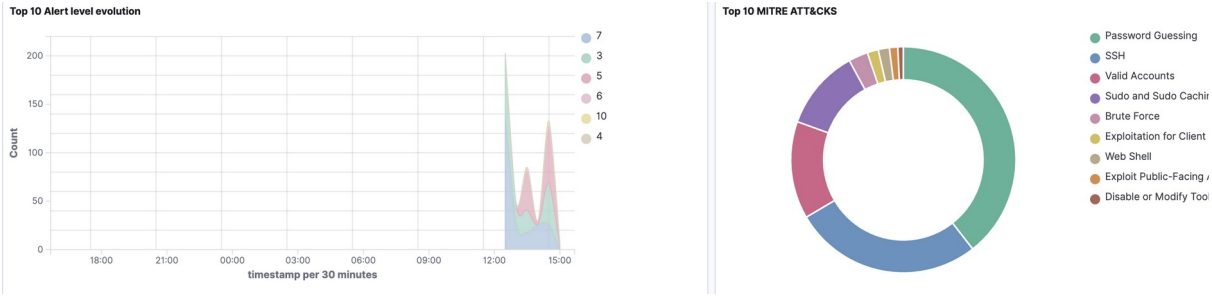


Figure 6b — Alert-level evolution and the Top MITRE ATT&CK techniques observed: Password Guessing / Brute Force, Web Shell, Valid Accounts, and more — the attack chain, classified automatically.

7. Detection Gaps & Recommendations

An honest assessment: the host-based SIEM covered the attack well from credential access onward, but two gaps are worth naming. Identifying them is part of the analyst's job.

7.1 Network reconnaissance was not detected

Observation. the Nmap scans that opened the attack produced no Wazuh alert. Wazuh is host-based (HIDS): it reads logs and watches files, but a port scan operates at the network layer and leaves little host-side log evidence to parse. (The UFW firewall did log the blocked scan traffic, but it did not rise to a correlated alert.)

Recommendation. add a network IDS — Suricata is the recommended choice — on the monitored segment and feed its alerts into Wazuh. Suricata inspects packets and would flag scans (e.g. ET SCAN signatures) and many on-the-wire exploits; its JSON output integrates natively with Wazuh, so network and host detections would appear in one dashboard. This closes exactly the gap seen here and is the basis for a natural follow-on project.

7.2 Command execution is detected by request, not by execution

Observation. rule 31514 fired because each command travelled in the URL query string, where Apache logs it. Wazuh saw the request carrying the command — not the PHP system() call actually executing. An attacker using a POST body, an encrypted channel, or an obfuscated shell would not expose the command in the web log, and this detection would miss it. The built-in rule also keys on the literal name shell.php, which a real attacker would simply rename.

Recommendation. deploy host command auditing (auditd execve logging) and ingest it into Wazuh, so command execution is detected at the point of execution regardless of how it was delivered. Complement filename-based rules with behavioural detection — e.g. the web-server user spawning a shell or unusual child processes — so detection does not depend on the attacker naming their shell shell.php.

8. Conclusion

The blue-team side of this exercise performed strongly. A multi-stage attack was detected across every post-reconnaissance phase, the entire kill chain was reconstructed from a single source-IP pivot, and the SSH brute force was automatically contained in real time by Active Response. File Integrity Monitoring independently caught the web shell at the moment it was written, demonstrating defence in depth.

Equally valuable are the two gaps identified: host-based monitoring does not see network reconnaissance, and request-based detection can be evaded. Both have concrete, standard remediations (Suricata NIDS integration; auditd execution auditing). Recognising these boundaries — rather than claiming total coverage — is what separates a monitoring deployment from a mature detection capability, and it sets the direction for the next project.

Methodology & Authorship Note

This report documents detection engineering and analysis I performed in a lab I built and own. Wazuh, its built-in rules, and the dashboards are standard platform features; the work represented here is the engineering and judgement around them: configuring the log sources, realtime FIM, and Active Response that produced these detections; running the investigation as an analyst would (pivot, scope, verify, reconstruct); mapping each stage to technique and impact; and critically assessing where the coverage falls short. The detection gaps in §7 are reported honestly because an accurate picture of what the SIEM does and does not see is more useful than an idealised one.